

PROGRAMISTOK 2026 • BIAŁYSTOK

# Inżynieria oprogramowania w dobie *agentów AI*

Hubert Łepicki

27 września 2026



# Witam!

„Stosowanie **systematycznego, zdyscyplinowanego i mierzalnego** podejścia do tworzenia, eksploatacji i utrzymania oprogramowania, czyli zastosowanie inżynierii do oprogramowania.”

IEEE Std 610.12-1990, *IEEE Standard Glossary of Software Engineering Terminology*

“The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.”



Margaret Hamilton przy wydrukach kodu oprogramowania lotu Apollo, 1969. Fot. Draper Laboratory (restauracja: Adam Cuerden), domena publiczna, Wikimedia Commons

HISTORIA • 1961–1972

# Margaret Hamilton i program Apollo

1961–72

program Apollo

20.07.1969

lądowanie Apollo 11

≈36k / 2k

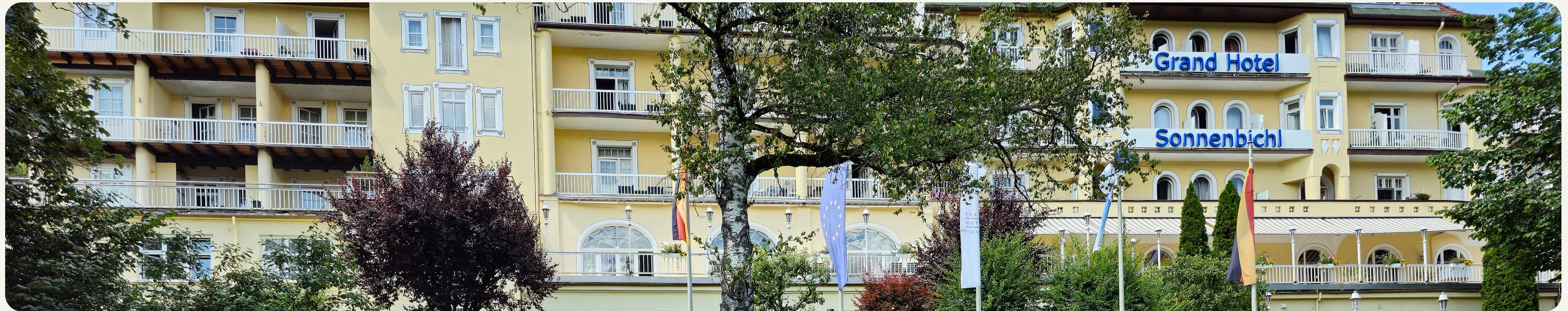
słów pamięci stałej /  
roboczej komputera AGC

*„Chciałam nadać naszemu oprogramowaniu  
prawomocność, żeby ono – i ci, którzy je tworzą –  
zyskali należny szacunek. Dlatego zaczęłam nazywać  
to, co robimy, inżynierią oprogramowania.”*

Margaret Hamilton, wywiad w: L. Snyder, R. Henry, *Fluency with Information Technology*, 7. wyd., Pearson 2017.

Oryginał: "I wanted to give our software 'legitimacy' so that it (and those building it) would be given its due respect; and, as a result I began to call what we were doing 'software engineering'..."

# NATO: dyscyplina dostaje nazwę



Hotel Sonnenbichl w Garmisch-Partenkirchen, miejsce konferencji z 1968 roku (zdjęcie z 2024). Fot. Ricardalovesmonuments, CC BY 4.0, Wikimedia Commons

01

**„Software crisis”**: duże systemy są spóźnione, drogie i zawodne.

02

**„Software engineering”** jako celowa prowokacja: to jeszcze nie jest inżynieria, ale powinno nią być.

03

**Efekt**: w kilka lat konferencja ICSE, czasopisma i przedmioty na uczelniach.

# Permanenty kryzys oprogramowania



## Błędy

dużo i wszędzie, często  
znajdowane dopiero na  
produkcji



## Terminy

przekraczane: „jeszcze  
tylko dwa tygodnie”



## Budżety

przekraczane, bo koszt  
zmiany rośnie z czasem

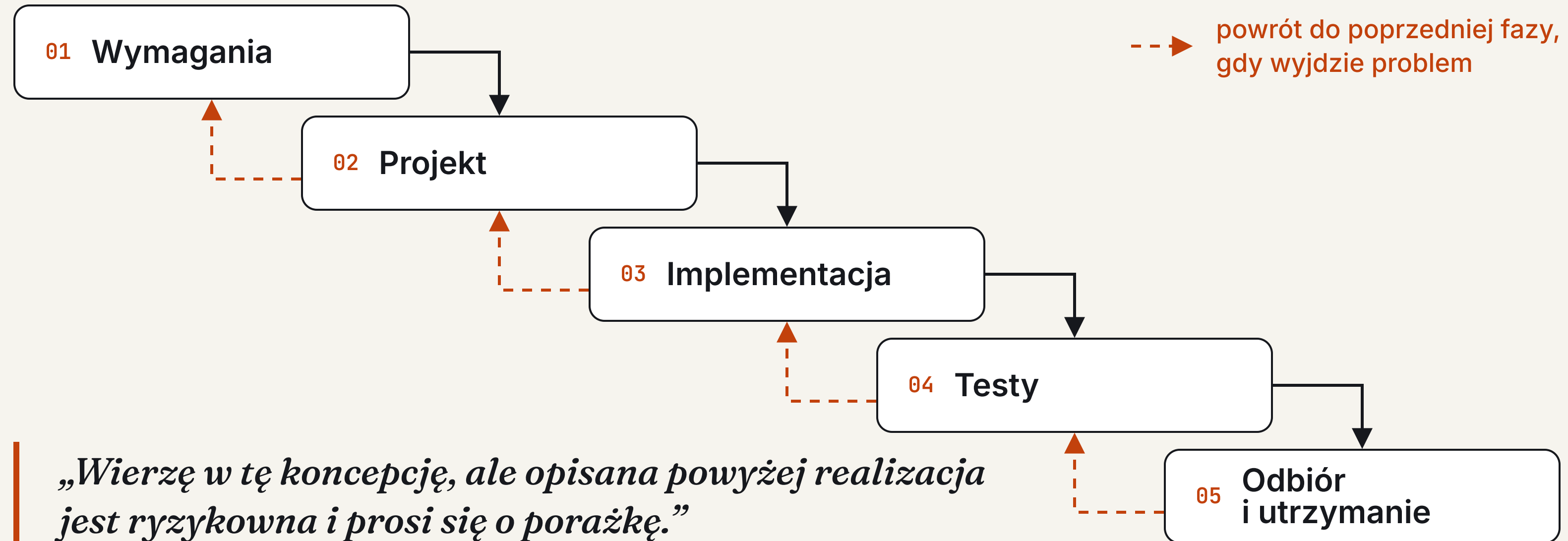


## Utrzymanie

kod, którego nikt nie  
rozumie i boi się  
zmieniać

Kryzys z 1968 roku *nigdy się nie skończył.*

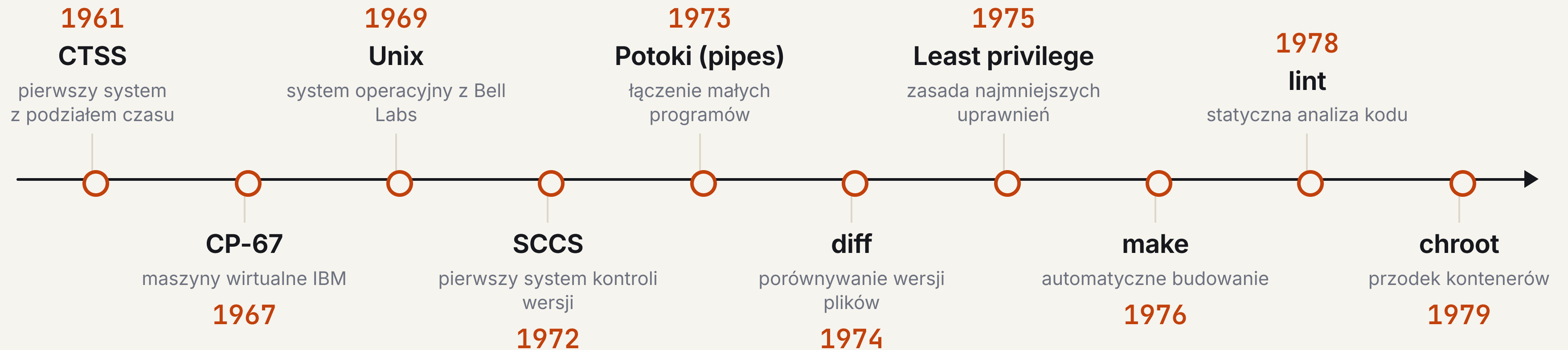
# Model kaskadowy



*„Wierzę w tę koncepcję, ale opisana powyżej realizacja jest ryzykowna i prosi się o porażkę.”*

Winston W. Royce, *Managing the Development of Large Software Systems*, 1970.  
Oryginał: "I believe in this concept, but the implementation described above is risky and invites failure."

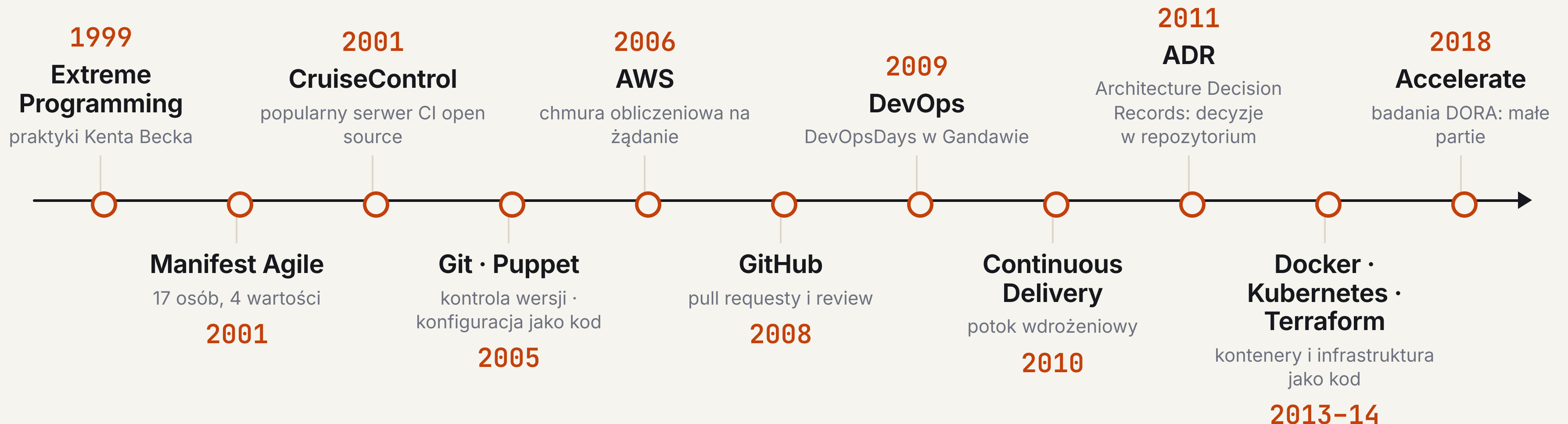
# Podział czasu, maszyny wirtualne, Unix, kontrola wersji



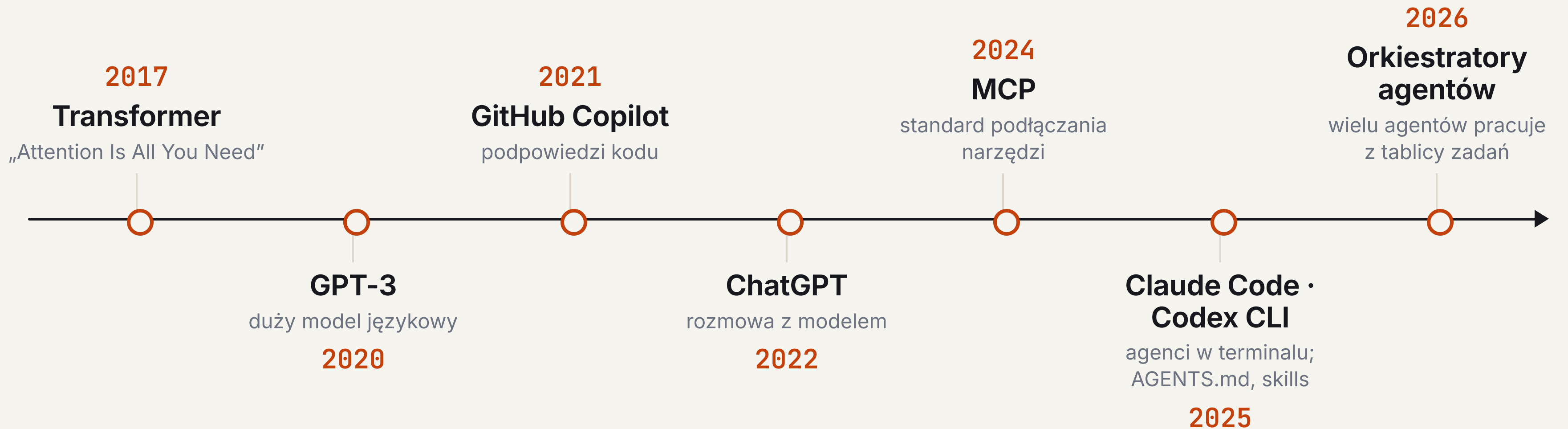
# Obiekty, wzorce projektowe, testy automatyczne



# Agile, DevOps, CI/CD, chmura, infrastruktura jako kod



# Modele językowe i agenci kodujący



# Inżynieria oprogramowania to dziś standard

Prawie każda firma i instytucja tworząca oprogramowanie ma swój zestaw praktyk. Trzymają się ich programiści, zespoły i całe organizacje, żeby dostarczać dobre oprogramowanie w budżecie i łatwe w utrzymaniu.

## PROGRAMISTA

- testy automatyczne, TDD
- refaktoryzacja
- standardy kodowania, lintery, formatery
- małe commity z opisem zmiany

## ZESPÓŁ

- Git, pull requesty, code review
- ciągła integracja (CI)
- Scrum albo Kanban, retrospektywy
- definicja ukończenia, dokumentacja

## ORGANIZACJA

- automatyczne wdrożenia (CD)
- infrastruktura jako kod, chmura, kontenery
- monitoring, dyżury, analiza awarii
- bezpieczeństwo: najmniejsze uprawnienia, sekrety, skanowanie zależności

## VIBE CODING

# Amnezja?

\* AI Agent

> zrób mi system sprzedaży biletów jak Eventbrite, ma działać bez błędów, szybko

- Jasne! Tworzę kompletny system sprzedaży biletów...
- Write(src/payments/processor.ts)  
 | ... i 62 kolejne pliki
- ✓ Utworzono 63 pliki, +14 382 linii. Gotowe! 🎉

... 3 miesiące później ...

\* Context compacted

> przecież ustaliliśmy, że płatności idą przez Stripe?

- Masz rację! Przepraszam za zamieszanie. Przepiszę moduł płatności od nowa...

# AI slop

## Add ticketing system #1337

↕ Open

vibe-coder wants to merge 1 commit into `main` from `feature/everything`

Conversation

Commits 1

Checks 1

Files changed 63

+14,382 -2,107

↗ update · a3f9c21

✖

CI / test — Failing after 4m

Required

!

Coverage — 12% of new lines covered

Informational

● reviewer

approved these changes · 3 minutes ago

LGTM 👍

Merge pull request ▼

- 14 tys. linii

w jednym PR-ze, jeden commit „update”
- 12% pokrycia

i czerwone CI
- „LGTM” po 3 minutach

nikt tego nie przeczytał

Inżynieria oprogramowania w dobie agentów AI · Programistok 2026

14 / 35

# O czym zapomnieliśmy?

## *O inżynierii oprogramowania.*

Systematyczne, zdyscyplinowane i mierzalne podejście do tworzenia, eksploatacji i utrzymania oprogramowania.

# Agent to nowy współpracownik

## PIERWSZEGO DNIA NIE ZNA

- naszych procedur i standardów
- projektu i jego architektury
- podjętych decyzji projektowych i ich powodów
- biznesowej strony projektu

Dlatego na początku często programujemy z nim  
w **parach** (pair programming).

## I KAŻDEGO DNIA MOŻE NIE PAMIĘTAĆ

Claude Code · Twój laptop

✓ lokalna pamięć, tylko na tym komputerze

ta sama sesja po streszczeniu kontekstu

~ szczegóły giną

inny komputer, inny programista

× nic nie wie o poprzednich sesjach

inna platforma: chmura, CI, inny agent

× nic nie wie o poprzednich sesjach

Wszystko, co ma wiedzieć, musi być zapisane tam, skąd *każda sesja* to przeczyta.

# Dostęp do dokumentacji, historii i kontekstu



## Dokumentacja

aktualizowana na bieżąco,  
razem z kodem



## Historia

w gicie pełna historia zmian  
i decyzji

numer ticketu w commicie łączy  
zmianę z jej powodem:

```
[#14] Rezerwuj miejsce na 15  
minut
```



## Kontekst i decyzje

decyzje projektowe w ADR-ach,  
częściowo w kodzie i historii  
gita, a zmiany decyzji  
w ticketach i PR-ach

przez

serwery MCP

gh CLI

API

AGENTS.md

prompts

skills

(więcej później)

## LEPSZY SPOSÓB PRACY Z AGENTEM

# Najpierw kontekst, potem kod

> Budujemy system sprzedaży biletów. Zadania są w [github.com/nasza-firma/bilety/issues](https://github.com/nasza-firma/bilety/issues), milestone „Faza 1”. Przeczytaj tickety i dokumentację, dopytaj o niejasności i przygotuj plan pierwszej fazy.

■ plan mode on (shift+tab to cycle)

- Bash(gh issue list --repo nasza-firma/bilety --milestone "Faza 1")
  - └ #12 Rejestracja · #13 Logowanie · #14 Kupno biletu · #15 Skanowanie przy bramce
- Bash(gh issue view 14 --comments)
  - └ „Płatności przez Stripe, bez przechowywania kart” (decyzja w komentarzu)
- Read(docs/architektura.md, docs/adr/)
  - └ 6 ADR-ów (Architecture Decision Records): wcześniejsze decyzje i ich powody
- Rozumiem cel: sprzedaż biletów na wydarzenia i wejście po zeskanowaniu kodu QR.
  - Faza 1 to cztery tickety, każdy jako osobny PR w stosie.
- Jedno pytanie: czy zwroty biletów wchodzą w fazę 1? W #14 nie ma decyzji.

> Zwroty dopiero w fazie 2.

## Plan fazy 1

1. #12 rejestracja → 2. #13 logowanie → 3. #14 kupno biletu (Stripe) → 4. #15 skanowanie  
Każdy krok: test akceptacyjny → TDD → review → PR w stosie

Would you like to proceed? 》 1. Yes 2. No, keep planning

- Zaczynam od #12: najpierw test akceptacyjny rejestracji.

## TAK JAK KORZYSTALIŚMY Z WIEDZY

# Dzielimy się wiedzą z ludźmi i agentami

```
> #14 gotowe. Podjęte decyzje projektowe zapisz w ADR-ach, zmiany względem oryginalnych wymagań opisz w tickecie, a znalezione istniejące błędy opisz jako nowe issues na GitHubie i dodaj do backlogu.
```

- `Write(docs/adr/0007-webhooki-stripe.md)`  
| decyzja: webhook Stripe może przyjść dwa razy, zapisujemy ID zdarzenia
- `Bash(gh issue comment 14 --body-file zmiany-wymagan.md)`  
| zmiana: miejsce rezerwowane na 15 minut do czasu płatności
- `Bash(gh issue create -t "Sesja nie wygasa po zmianie hasła" -l bug -p Backlog)`  
| #33 w backlogu: istniejący błąd w logowaniu, kroki do odtworzenia
- `Bash(gh pr create --title "Kupno biletu (#14)" --body-file pr.md)`  
| PR #31, linki: #14 ↔ ADR 0007 ↔ #33

### ADR

podjęta decyzja projektowa i jej powód

### Ticket

zmiany względem oryginalnych wymagań

### Nowe issues

znalezione błędy, od razu w backlogu

### Pull request

łączy wszystko linkami

Następna sesja, recenzent i człowiek przy poprawkach *dostają cały kontekst.*

# Powiedzmy agentowi, gdzie jest wiedza

AGENTS.md

## # Gdzie szukać wiedzy

- Zadania i decyzje: GitHub Issues,  
`gh issue view <nr> --comments`
- Architektura: `docs/architektura.md`
- Decyzje projektowe: `docs/adr/`
- Historia: `git log`, opisy PR-ów

## # Jak dzielić się wiedzą

- Decyzja projektowa → nowy ADR
- Zmiana wymagań → komentarz w tickecie
- Znaleziony błąd → issue `bug` w Backlogu
- Procedura: skill `zapis-wiedzy`

**Zawsze w kontekście:** krótka mapa, gdzie co jest.

skills/zapis-wiedzy/SKILL.md

---

**name:** zapis-wiedzy

**description:** Użyj po zakończeniu zadania  
albo przy decyzji projektowej.

---

1. Decyzja? Skopiuj `docs/adr/szablon.md`,  
nadaj kolejny numer.
2. Zmiana wymagań? `gh issue comment <nr>`  
z sekcją „Zmiana względem zadania”.
3. Błąd poza zakresem?  
`gh issue create -l bug -p Backlog`
4. PR: Decyzje • Problemy • Zmiany • Linki

**Wczytywany, gdy potrzebny:** dokładna procedura.

# Commy, które mówią dlaczego

AGENTS.md

```
# Git
- Gałęzie: feature/<nr-ticketu>-krotki-opis
- Temat commita zaczynaj od [#nr-ticketu]
- W treści: co się zmienia i dlaczego
- Na końcu: Co-authored-by z nazwą agenta
- Commy tylko przez skill git-commit
```

.agents/skills/git-commit/SKILL.md

```
---
name: git-commit
description: Commit po sprawdzeniu formatera
i akceptacji człowieka.
---
1. Uruchom formater. Coś zmienił? Zapytaj.
2. Przejrzyj git diff i git log --oneline.
3. Temat: [#nr] + opis, treść: dlaczego.
4. Pokaż wiadomość i pliki, czekaj na zgodę.
5. Dodaj tylko zamierzone pliki i commituj.
```

git log

```
commit 8e41c07 (feature/14-kupno-biletu)
[#14] Rezerwuj miejsce na 15 minut do czasu płatności

Bez rezerwacji dwie osoby mogły kupić to samo miejsce.
Decyzja w docs/adr/0007, zmiana zakresu opisana w #14.

Co-authored-by: AI Agent <agent@nasza-firma.dev>
```

# Konfiguracja agentów to kod zespołu

## W REPOZYTORIUM PROJEKTU

```
bilety/  
├─ AGENTS.md  
├─ CLAUDE.md → AGENTS.md  
├─ skills/zapis-wiedzy/  
└─ docs/adr/
```

## W OSOBNYM REPOZYTORIUM ZESPOŁU

```
nasza-firma/agent-plugins  
├─ plugins/tdd/           skills, hooki  
├─ plugins/code-review/  
└─ plugins/zapis-wiedzy/
```



## WSZYSCY AGENCI ZESPOŁU

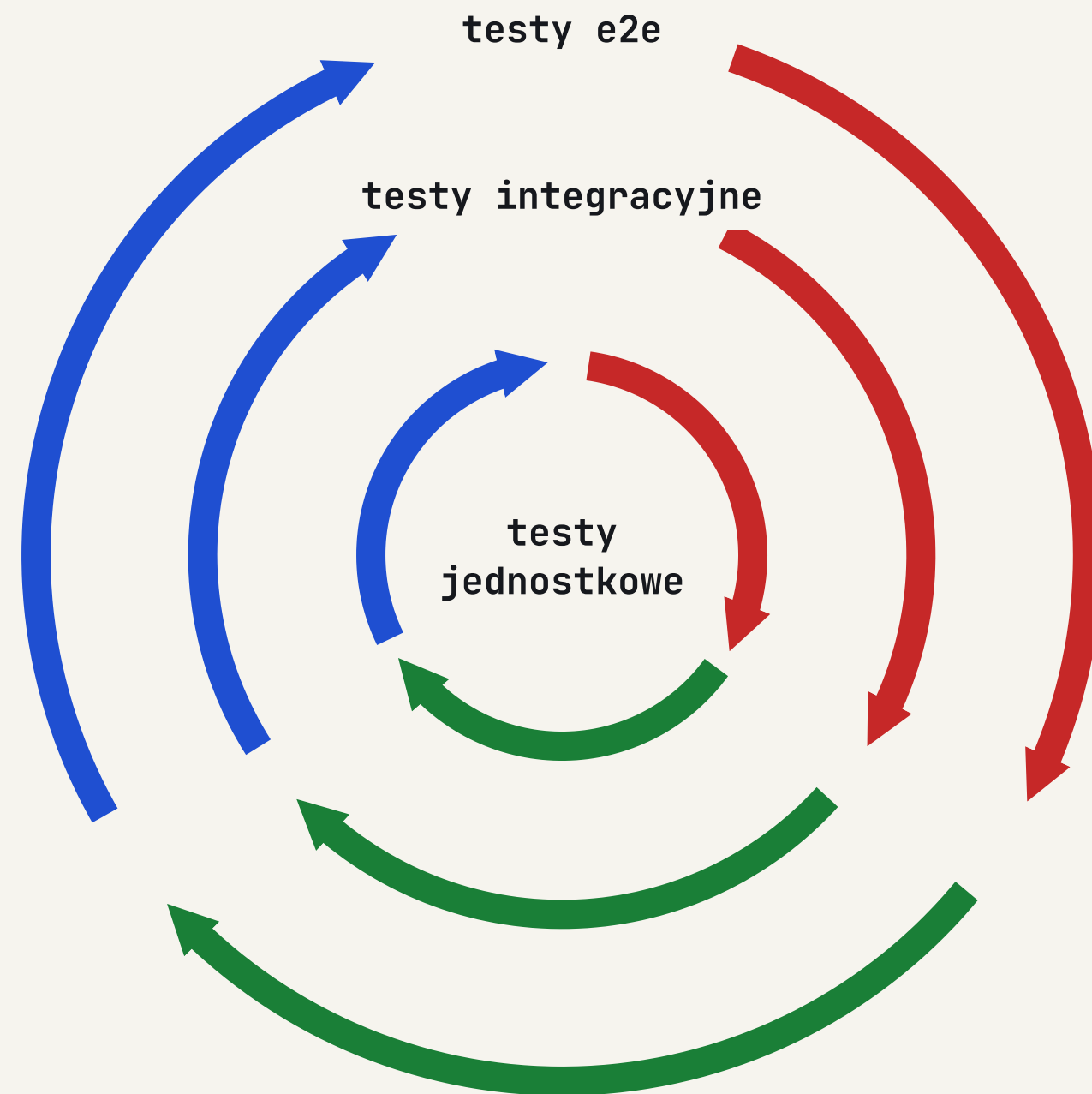
- agent na Twoim laptopie
- agent koleżanki z zespołu
- agent w CI robiący review
- agent w chmurze, z tablicy zadań

✓ te same techniki i metody

- Zmiany w konfiguracji przez **pull request i review**, jak każdy kod.
- Wersjonowana razem z projektem albo jako **plugin instalowany przez cały zespół**.

↩ Przykład pluginu: [github.com/hubertlepicki/agile](https://github.com/hubertlepicki/agile)

# TDD: przypomnienie



- RED test, który nie przechodzi, z właściwego powodu
- GREEN najmniej kodu, żeby przeszedł
- REFACTOR porządki przy zielonych testach

---

## Pętla zewnętrzna: testy e2e

cała ścieżka użytkownika, od przeglądarki po bazę

## Pętla środkowa: testy integracyjne

współpraca modułów, baza danych, zewnętrzne API

## Pętla wewnętrzna: testy jednostkowe

pojedyncze zachowanie, szybko i w małych krokach

# TDD zapisane dla agenta

AGENTS.md

## # Sposób pracy

- Każda zmiana zachowania zaczyna się od testu, który nie przechodzi.
- Od zewnątrz do środka: e2e → integracyjne → jednostkowe.
- Nie usuwaj, nie pomijaj i nie osłabiaj testów, żeby przeszły.
- Dokładny cykl: `skill tdd`

Zawsze w kontekście: zasada i odsyłać.

skills/tdd/SKILL.md

---

**name:** tdd**description:** Użyj przy każdej zmianie zachowania.

---

1. Z ticketa wypisz zachowania jako przykłady.
2. Test e2e: uruchom, sprawdź, że pada z właściwego powodu.
3. Zejdź niżej: integracyjny, jednostkowy. Każdy najpierw **RED**.
4. **GREEN**: najmniej kodu, żeby przeszedł.
5. **REFACTOR** kodu i testów przy zielonych.
6. Wracaj w górę, aż przejdzie test e2e.
7. Commit na zielonym: `skill git-commit`.

Wczytywany, gdy potrzebny: dokładny cykl.

# Agent sprawdza zmianę jak użytkownik

- Dobry programista, zanim odda zmianę, **zawsze przeklika ją ręcznie**. Samo TDD to za mało.
- Agent też to potrafi: **computer use, Chrome DevTools, Playwright**. Widzi to, co użytkownik: ekran, konsolę, ruch sieciowy.
- Znaleziony błąd to **nowy czerwony test** i powrót do cyklu TDD.

skills/test-uzytkownika/SKILL.md

---

**name:** test-uzytkownika

**description:** Użyj przed oddaniem zadania.

---

1. `gh issue view <nr>`: wypisz przypadki użycia i kryteria akceptacji.
2. Uruchom aplikację, otwórz ją przez `Playwright` albo `Chrome DevTools`.
3. Przejdź każdy przypadek jak użytkownik: klikaj, wpisuj, patrz na ekran i konsolę.
4. Zrób zrzuty ekranu jako dowód.
5. Błąd? Najpierw test, który go odtwarza (RED), potem poprawka (GREEN), refaktor.
6. Wyniki i zrzuty dołącz do PR.

← Apollo: najpierw misja w symulatorze

# Wymuszamy proces pluginem

## PROBLEM

- Reguła „najpierw test” w AGENTS.md / CLAUDE.md **jest w kontekście, ale w długiej sesji agent coraz częściej ją pomija**, zwłaszcza po streszczeniu kontekstu.
- Przypominanie o niej w każdym prompcie jest **uciążliwe** i łatwo o nim zapomnieć.

## ROZWIĄZANIE

- Pakujemy naszą logikę w **plugin** i korzystamy z mechanizmów agenta, np. **hooków**.

### PRZYPOMINAJĄ REGUŁY

**SessionStart** pełne reguły na starcie i po streszczeniu kontekstu

**UserPromptSubmit** krótkie przypomnienie przy każdym prompcie

**SubagentStart** pełne reguły dla każdego subagenta

### PILNUJĄ DZIAŁAŃ

**PreToolUse** blokuje ryzykowne działanie, np. push na *main* albo commit bez [#nr]

**PostToolUse** po edycji uruchamia formater i testy, błędy wracają do agenta

**Stop** nie pozwala skończyć, dopóki testy nie przechodzą i nie sprawdzono przypadków użycia z perspektywy użytkownika



<https://github.com/hubertlepicki/agile>

*Talk first. Test first.*

*Small steps. Plain words.*

Claude Code · Codex · OpenCode · Antigravity · Grok Build

# Kiedy potrzebny jest plugin?

01

## Regułę trzeba wymusić

hooki działają zawsze, niezależnie od tego, czy model pamięta

02

## Wiele elementów działa razem

skill, subagent, komenda i hook w jednej paczce

03

## Wiele repozytoriów

instalacja raz, wersje, aktualizacja dla całego zespołu

04

## Integracje

serwery MCP (Jira, Linear) i LSP, czyli rozumienie kodu

05

## Subagenci z uprawnieniami

recenzent tylko do odczytu, tester tylko z przeglądarką

06

## Workflow uruchamia człowiek

komendy `/review`, `/commit` działają zawsze na żądanie

---

**AGENTS.md** co i gdzie   **skills** jak   **plugin** dopilnować i rozdać zespołowi

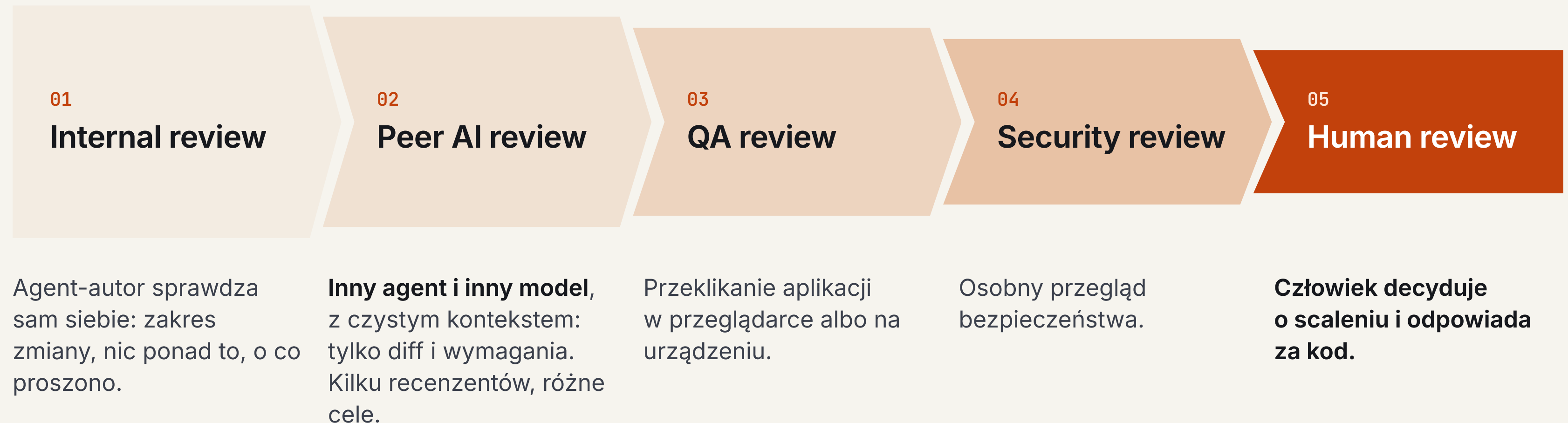
# Kontrola wersji i stosy pull requestów

- **Pamiętacie AI slop?** Agent w godzinę wygeneruje kilkanaście tysięcy linii w jednym PR-ze. Nikt tego rzetelnie nie przejrzy.
- **Iteracyjne zmiany** i częste commity z opisem *dlaczego*, nie tylko *co*.
- **Małe PR-y:** jedna zmiana, jeden cel. Najpierw refaktoryzacja bez zmiany zachowania, potem zmiana zachowania.
- **Stos PR-ów:** każdy buduje na poprzednim i każdy da się przejrzeć osobno.
- Rebase/merge stosu to dla agenta **tania, mechaniczna praca**.



**GitHub, lipiec 2026:** natywne stosy PR-ów (public preview) i rozszerzenie `gh stack`

# Wielopoziomowy code review



↩ Fagan, IBM 1976: role i listy kontrolne, dziś zapisane w prompcie

# Od „Enter, Enter, Enter” do trybu YOLO

01

**Potwierdzam każdą komendę**

czytam, rozumiem, zatwierdzam

02

**Udaję, że czytam**

maszynka do naciskania Entera

03

**Auto-approval, tryb YOLO**

agent robi, co uzna za słuszne

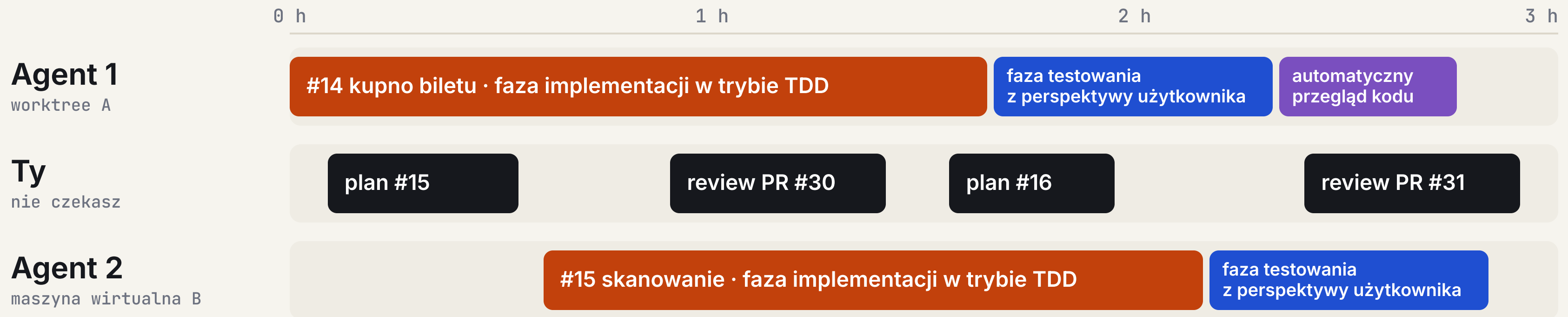
## AGENT BEZ NADZORU MOŻE

- nie odtworzyć błędu lokalnie i **spróbować na produkcji**
- zajrzeć do **produkcyjnej bazy**, żeby sprawdzić dane
- w Twoim shellu mieć **Twoje uprawnienia**: `kubectl`  
`gh` `git push` `ssh`

## DLATEGO

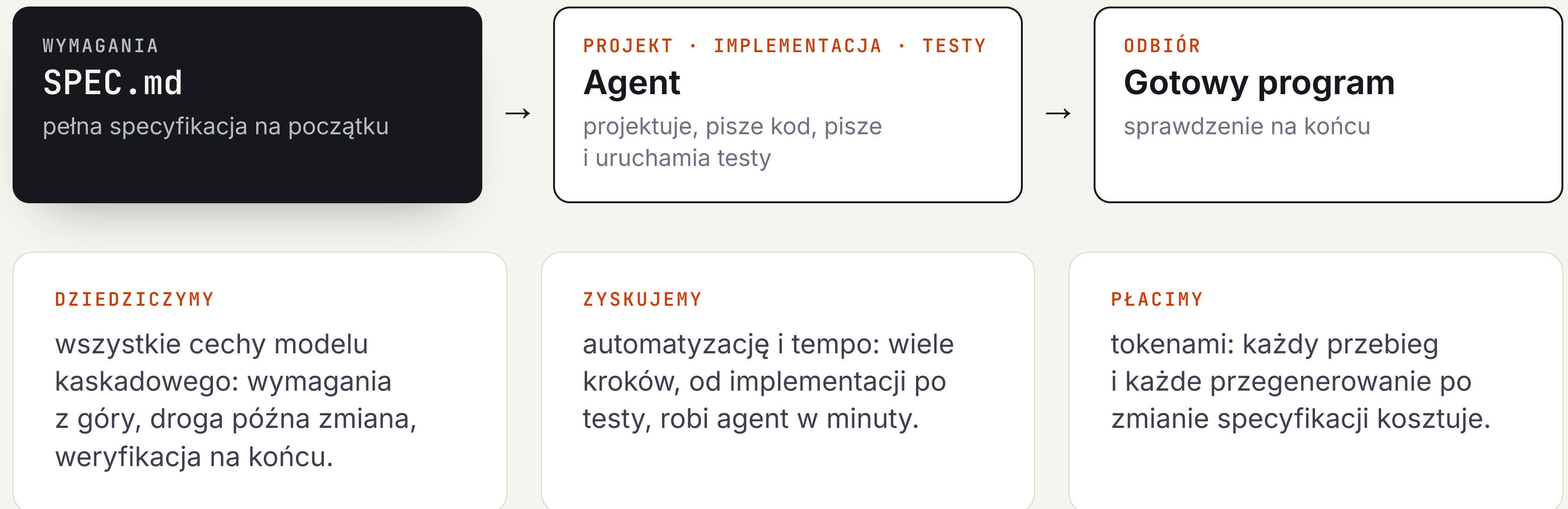
- **Najmniejsze uprawnienia**: bez produkcji i produkcyjnych sekretów
- **Sandbox**: kontener albo maszyna wirtualna, ograniczona sieć
- **Chroniony main**: scalenie i wdrożenie tylko po akceptacji człowieka

# Długie sesje: nie czekajmy na agenta



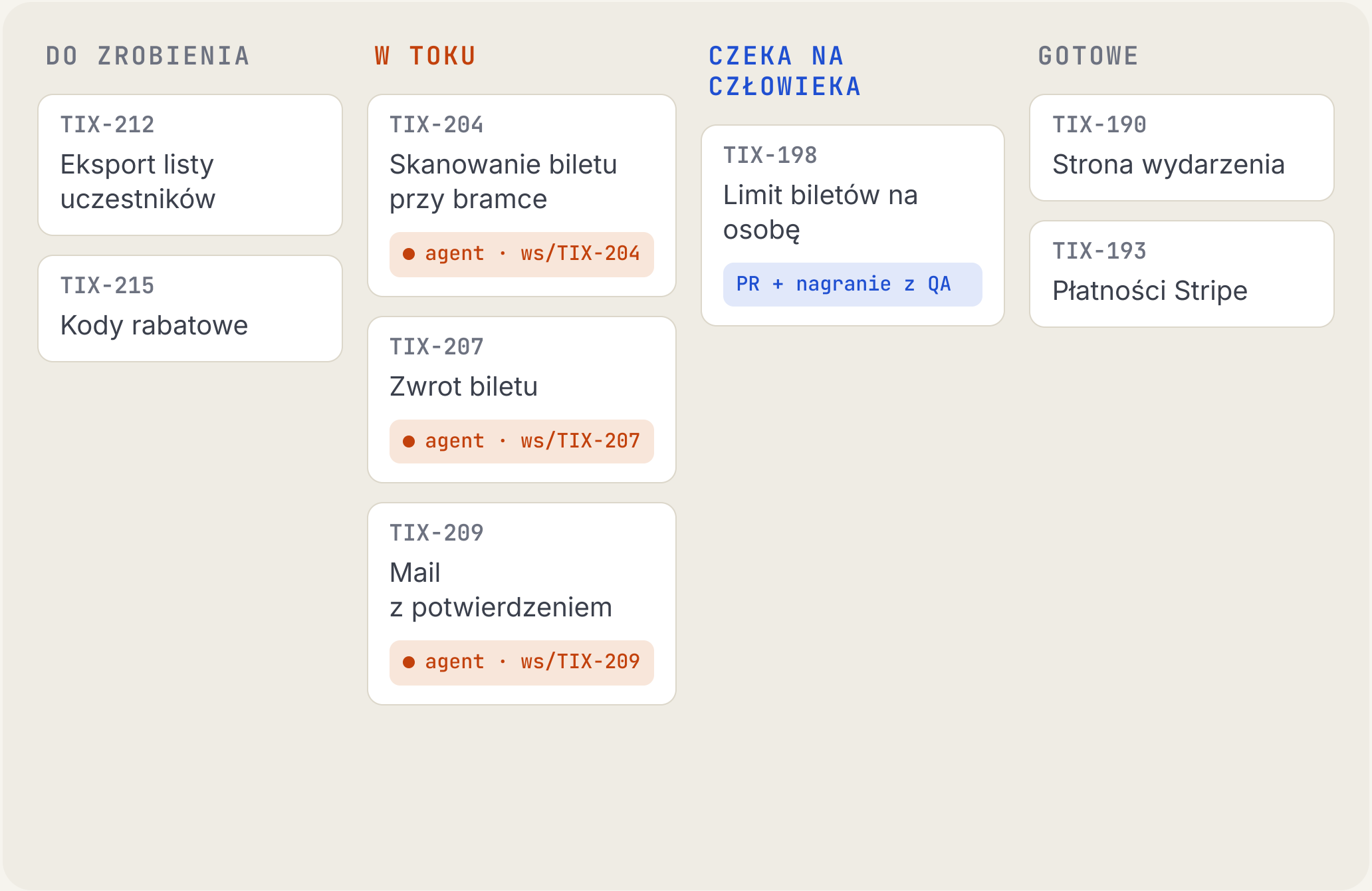
- Zadanie w trybie TDD z testem ręcznym to **nawet kilka godzin** pracy agenta.
- Nie czekamy: **przeglądamy kod, planujemy** kolejne zadania.
- **Git worktrees i maszyny wirtualne:** druga instancja agenta pracuje nad innym zadaniem. VM to przy okazji dobry sandbox.

# Spec-driven development



↔ Royce 1970: wymagania → projekt → implementacja → testy → odbiór

# Orkiestrator autonomicznych agentów



- Tablica zgłoszeń (Linear) jako panel sterowania.
- Każde zgłoszenie dostaje **własnego agenta we własnym katalogu**.
- **WORKFLOW.md w repozytorium:** stany, przejścia, kiedy czekać na człowieka, jaki dowód wykonania zostawić.

```
WORKFLOW.md

---
tracker:
  kind: linear
  active_states: [Todo, In Progress]
agent:
  max_concurrent_agents: 3
---

Pracujesz nad {{ issue.identifier }}.
Gdy skończysz, otwórz PR z dowodem
wykonania i przenieś zgłoszenie
do „Human Review”.
```

# Inżynieria oprogramowania w pracy z agentami

## KONTEKST I WIEDZA

- tickety, ADR-y, dokumentacja przez `gh` i MCP
- tryb planowania: pytania przed kodem
- AGENTS.md i skills: gdzie szukać, jak pracować
- zapis decyzji, zmian wymagań i znalezionych błędów

## SPOSÓB PRACY

- TDD od zewnątrz do środka
- test z perspektywy użytkownika w przeglądarce
- commity z `[#nr]` i opisem dlaczego
- małe PR-y w stosie

## KONTROLA JAKOŚCI

- wielopoziomowy review: inny agent, inny model
- człowiek decyduje o scaleniu
- hooki i pluginy wymuszają proces
- linter, CI, merge queue

## BEZPIECZEŃSTWO I SKALA

- najmniejsze uprawnienia, sandbox
- chroniony `main`
- worktrees i maszyny wirtualne do pracy równoległej
- konfiguracja agentów wspólna dla zespołu

Inżynieria oprogramowania jest dziś *ważniejsza niż kiedykolwiek*.

# Dziękuję

@hubertlepicki